

Trabajo Práctico Integrador –

Programación I

Carrera: Tecnicatura Universitaria en

Programación

Materia: Programación I

Integrantes:

Facundo Nicolás Martino

– Julián Campanini

Año: 2025

Índice

1. Introducción
2. Marco Teórico
3. Tecnologías y Herramientas Utilizadas
4. Relación entre la teoría y el proyecto
5. Conclusión
6. Bibliografía

1. Introducción El presente trabajo tiene como objetivo desarrollar un sistema de gestión de países utilizando Python. El proyecto integra conceptos fundamentales de la materia Programación I, tales como listas, diccionarios, funciones, condicionales, manejo de archivos y ordenamientos. También incorpora herramientas avanzadas como manejo de excepciones, expresiones lambda y comprensión de listas, cuya fundamentación se desarrolla en este informe según pedido del docente

2. Marco Teórico

Listas en Python

Una lista es una estructura de datos que permite almacenar múltiples elementos ordenados y modificables. Se utilizan ampliamente para gestionar colecciones de datos. En este proyecto, las listas permiten almacenar todos los países cargados desde el archivo CSV, lo que permite recorrerlos, filtrarlos y ordenarlos. Ejemplo de uso: `lista.append(nuevo_país)`.

Diccionarios

Un diccionario almacena pares clave-valor. Es ideal para representar entidades con atributos. En el proyecto, cada país se representa como un diccionario con claves NOMBRE, POBLACION, SUPERFICIE y CONTINENTE. Esto facilita acceder a valores específicos y ordenarlos según criterios personalizados.

Funciones

Permiten dividir un problema en partes más pequeñas, mejorar la claridad del código y evitar repeticiones. El proyecto cuenta con funciones para agregar países, buscar, actualizar, ordenar, guardar datos, etc. La modularización es una buena práctica fundamental en el desarrollo de software.

Condicionales y estructuras de control

El uso de `if/elif/else` permite tomar decisiones según condiciones dadas por el usuario o los datos. El menú principal emplea condicionales para interpretar la opción ingresada.

Lectura y escritura de archivos CSV

Se utiliza `open()` junto con modos como `a+` y `w` para permitir tanto leer como escribir datos. El archivo CSV garantiza persistencia, es decir, que la información permanezca disponible aunque el programa termine.

Ordenamientos

Se utiliza la función `sort()` con claves personalizadas y lambdas para ordenar la lista de países por distintos criterios. Las funciones lambda permiten definir pequeños comportamientos inline sin necesidad de funciones completas.

Funciones lambda

Una función lambda es una función pequeña y anónima que se escribe en una sola línea. Se utiliza para operaciones simples en las que definir una función completa sería innecesario.

¿Cómo funciona una lambda?

La sintaxis es:

```
lambda parámetros: expresión
```

Una lambda siempre devuelve el valor de la expresión. Por ejemplo:

```
dobles = lambda x: x * 2  
print(dobles(5)) # Resultado: 10
```

Uso en el proyecto

Las lambdas se usan para ordenar la lista de países. Por ejemplo:

```
lista.sort(key=lambda p: p['POBLACION'])
```

Aquí `lambda p: p['POBLACION']` indica que el criterio de ordenamiento es la población del país. Esto permite ordenar con una sola línea de código. Sin lambdas, habría que definir una función aparte.

Manejo de excepciones (try/except)

El manejo de excepciones evita errores fatales cuando se trabaja con archivos u operaciones que puedan fallar. Es una herramienta avanzada no presentada en la cursada, pero utilizada para mejorar la robustez del programa. Su uso está justificado porque permite manejar situaciones como archivo inexistente o líneas corruptas en el CSV.

Los bloques try/except permiten manejar errores (excepciones) que ocurren durante la ejecución del programa. Sin este mecanismo, si ocurre un error, por ejemplo, intentar abrir un archivo inexistente, el programa se detendrá inmediatamente. Con try/except podemos anticiparnos a esos errores y decidir cómo reaccionar.

¿Cómo funciona try/except?

La estructura básica es:

```
try:
    # Código que puede generar un error
except:
    # Código que se ejecuta si ocurre un error
```

El código dentro de try se ejecuta normalmente. Si ocurre una excepción, Python detiene la ejecución del bloque try y salta al except correspondiente.

Ejemplo aplicado al proyecto

En la función cargar_paises(), el programa intenta abrir y leer un archivo CSV. Si el archivo no existe o está dañado, sin try/except el programa fallaría. Con try/except, simplemente se informa el error al usuario:

```
try:
    with open('paises.csv', 'a+') as f:
        ...
except Exception as e:
    print('Error al leer el archivo:', e)
```

Esto hace que el programa sea más robusto y amigable. Además, permite manejar casos inesperados como líneas corruptas o problemas de permisos.

Comprensión de listas

La comprensión de listas es una forma compacta y eficiente de crear listas nuevas a partir de otras listas. Permite reemplazar bucles for completos con una sintaxis clara y directa.

Ejemplo básico

```
nums = [1, 2, 3, 4]

cuadrados = [n * 2 for n in nums]
```

Esto genera una nueva lista sin necesidad de escribir un bucle. Es más rápido y más fácil de leer.

Uso en el proyecto

La comprensión de listas se utiliza para filtrar países según el continente:

```
resultados = [p for p in lista if p['CONTINENTE'].lower() == cont]
```

Este código reemplaza un bucle tradicional como:

```
resultados = []  
  
for p in lista:  
  
    if p['CONTINENTE'].lower() == cont:  
  
        resultados.append(p)
```

El uso de comprensión de listas hace que el código sea más conciso, expresivo y Pythonic

3. Relación entre la Teoría y el Proyecto

Cada concepto teórico explicado anteriormente se refleja directamente en el sistema desarrollado. El almacenamiento de información se realiza mediante listas y diccionarios. Las funciones permiten dividir el sistema en subsistemas como carga, búsqueda y ordenamiento. El manejo de errores permite asegurar que el programa no falle ante datos corruptos. El uso de lambdas y comprensión de listas mejora la eficiencia y claridad del código, justificando su incorporación aun cuando excedan los contenidos mínimos de la materia.

4. Conclusión

El proyecto integra correctamente los contenidos fundamentales de Programación I y demuestra la capacidad de los estudiantes para aplicar conceptos teóricos en un caso práctico. Asimismo, se incorporan herramientas más avanzadas para mejorar la calidad y robustez del código, justificadas adecuadamente en este informe.

5. Bibliografía - Python Documentation. <https://docs.python.org/>

- Bibliografía UTN San Nicolas.